Gemini CLI: Concrete Architecture & Reflexion Analysis

Omar Afify, 20287159
Dylan Spilberg, 20266708
Rowan Mohammed, 20233462
Muhammad Ibrahim, 20273315
Keven Li, 20322014
Zonghan Li, 20321365
Queen's University, Computer Engineering
CMPE 322 Software Architecture

March 13, 2026

Abstract

This report presents a comprehensive analysis of the concrete architecture of the Gemini CLI system, bridging the gap between theoretical conceptual models (Assignment 1) and actual implementation realities. Utilizing package boundary analysis and import graph tracing, the system's concrete architecture is mapped into seven distinct, horizontally layered components, fundamentally challenging previous assumptions of a strict Client/Server hierarchy. A rigorous analysis into the Tool Execution Framework reveals a unidirectional, hub-and-spoke topology orchestrated by a central `Scheduler` and secured by a non-bypassable `PolicyEngine`. Through concrete execution traces of both read-only and safety-gated write operations, this document demonstrates how the system reliably and securely manages non-deterministic AI outputs. Finally, a detailed reflexion analysis highlights critical architectural divergences, most notably the necessity of a first-class Security subsystem, and outlines key engineering lessons learned regarding defense-in-depth orchestration and the realities of AI-assisted codebase exploration.

Contents

1 Introduction & Architecture Derivation **2**
1.1 Architecture Derivation Process . . . 2

2 High-Level Concrete Architecture **3**
2.1 Reference: Conceptual Architecture (A1) . . . 3
2.1.1 Rationale for Updating A1 . . . 3
2.2 Concrete Architecture Diagram . . . 4
2.3 Component Breakdown . . . 6
2.3.1 Terminal Interface . . . 6
2.3.2 Orchestration Engine . . . 6
2.3.3 Tool Execution Framework . . . 6
2.3.4 External Integration Layer . . . 6
2.3.5 Task Delegation System . . . 7
2.3.6 Security & Execution . . . 7
2.3.7 Config System . . . 7
2.4 Architectural Dependencies . . . 7

3 Tool Execution Framework **8**
3.1 Conceptual Architecture . . . 8
3.2 Concrete Architecture . . . 9
3.3 Sub-Component Interactions . . . 9
3.4 Concrete Dependency Analysis . . . 10

4 Use Cases and Execution Traces **11**
4.1 Use Case 1: Read-Only Tool Execution (`read_file`) . . . 11
4.2 Use Case 2: Safety-Gated Code Modification (`write_file`) . . . 12

5 Reflexion Analysis **14**
5.1 High-Level Architectural Divergences . . . 14
5.1.1 Client/Server Split vs. Layered Monorepo . . . 14
5.1.2 Strict Layered Stack vs. Bypassed Mediation . . . 14
5.1.3 Implicit Security vs. Entangled First-Class Subsystems . . . 14
5.1.4 Tightly Coupled Orchestration and Tool Execution . . . 15
5.1.5 Confirmed Architectural Boundaries (Unchanged from A1) . . . 15
5.2 Subsystem Reflexion: Tool Execution Framework. . . . 15
5.3 Summary of Rationales . . . 16

6 Lessons Learned **16**

A AI Collaboration Report **16**

1 Introduction & Architecture Derivation

The purpose of this report is to analyze and document the concrete architecture of the Gemini CLI system, providing insight into its structural organization and operational behavior. Building upon our earlier conceptual models [1, 2], this analysis uncovers the actual implementation of the system's foundational components and evaluates how closely the deployed codebase aligns with the initial architectural assumptions.

1.1 Architecture Derivation Process

The mapping from the raw source code to our reference architecture components was derived through a systematic, iterative process. We assigned code files and folders from the codebase to the conceptual components defined in Assignment 1 using the following methodologies:

- **Package Boundary Analysis:** By examining the monorepo structure, we observed a strict split between `packages/cli` (the user-facing terminal UI) and `packages/core` (the backend processing logic). This boundary immediately and cleanly separated the Terminal Interface from all other service-layer components.
- **Import Graph Tracing:** Starting from the main application entry point (`packages/cli/src/ gemini.tsx`), we traced `import` statements across modules. This process revealed the underlying data and control flow, helping us accurately identify which modules depend on one another.
- **Responsibility Grouping:** Files were systematically grouped according to their primary architectural responsibility. For example, logic dedicated to evaluating policies was grouped separately from scheduling tool execution or communicating with the Gemini API. These functional groups were then mapped to the most appropriate reference component.
- **Naming Convention & Directory Structure:** The mapping was further reinforced by the codebase's descriptive directory names (e.g., `tools/`, `agents/`, `mcp/`, `safety/`, `scheduler/`), which strongly correlated with and naturally mapped to our reference architecture components.

2 High-Level Concrete Architecture

This section presents the concrete system-level architecture of Gemini CLI, derived from static dependency analysis using SciTools Understand. We reference the conceptual architecture established in A1, describe the concrete architecture as a box-and-arrow diagram with subsystem dependencies, and provide a reflexion analysis of the key divergences between the two.

2.1 Reference: Conceptual Architecture (A1)

The conceptual architecture from A1 (Figure 1) organized the system into a layered topology:

- **Application Layer** containing the *Terminal Interface* (user-facing React/Ink UI) and the *Orchestration Engine* (core chat loop, LLM client, prompt construction, model routing).
- **Service Layer** (nested within the Application Layer) containing the *Tool Execution Framework*, *Task Delegation System*, and *External Integration Layer*.
- **Cross-cutting components**: *Security & Execution* (policy engine, confirmation bus, safety checkers) and *Config System* (settings, hooks, extensions, credentials).
- **External boundaries**: Gemini API, MCP Servers, and the A2A Server.

The conceptual model assumed **strict top-down dependency flow**: the Terminal Interface drives the Orchestration Engine, which exclusively mediates access to the Service Layer. Cross-cutting components were expected to be passively consumed by all layers without creating reverse dependencies.

2.1.1 Rationale for Updating A1

In A1, the focus is more high-level to identify major sub-systems in the program, such as CLI frontend, core engine and the tool system. However, there is no clear separation of internal responsibilities within the system. The improved diagram is also more organized as it classifies components into Application and Service layers. This design choice better reflects the real separation of concerns in the system and allows for more traceability when modeling code components. On the other hand, more details are added into the diagram for more in-depth comparison. For example, the tool manager is now framed as a framework rather than just a service, or a sub-system component. This shows that the external tools are managed by the framework instead of being called directly by the core. Also, some previously not-shown abilities, such as external integration, task delegation, and the security gate are now all modelled explicitly. Another major upgrade made to the diagram is that the external systems and the internal components are now separated with a clear boundary. Overall, those upgrades make this conceptual architectural diagram more valuable for A2.

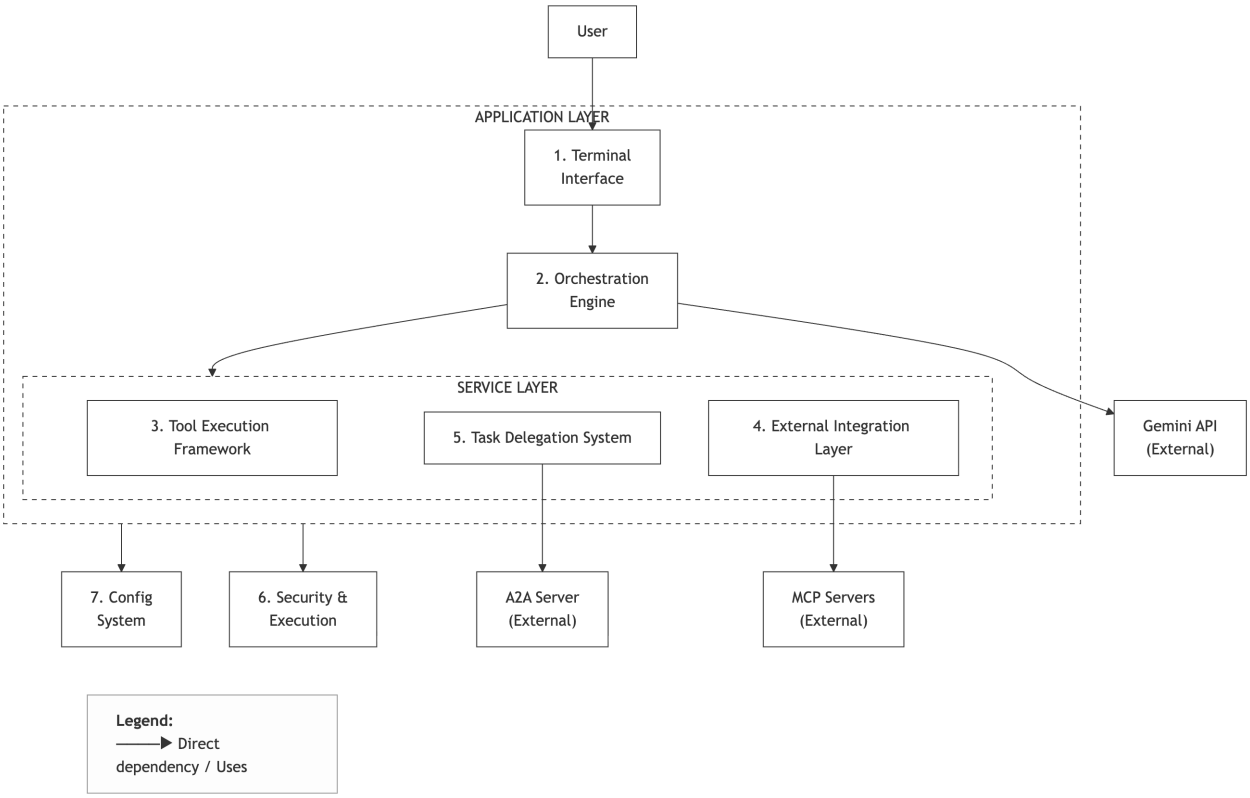

Figure 1: Conceptual system architecture from A1. Arrows indicate dependency direction (caller → callee).

2.2 Concrete Architecture Diagram

To derive the concrete architecture, we assigned all non-test source files across `packages/cli`, `packages/core`, and `packages/a2a-server` to the seven components defined in A1, then generated a dependency graph using SciTools Understand (Figure 2).

Figure 2: Dependency graph from SciTools Understand. Red edges indicate bidirectional coupling; blue edges are unidirectional. Edge labels show import counts (*forward / reverse*).

From this analysis, we derived the concrete architecture diagram (Figure 3), which preserves the conceptual model's component boundaries but updates the dependency edges to reflect the actual couplings discovered through static analysis.

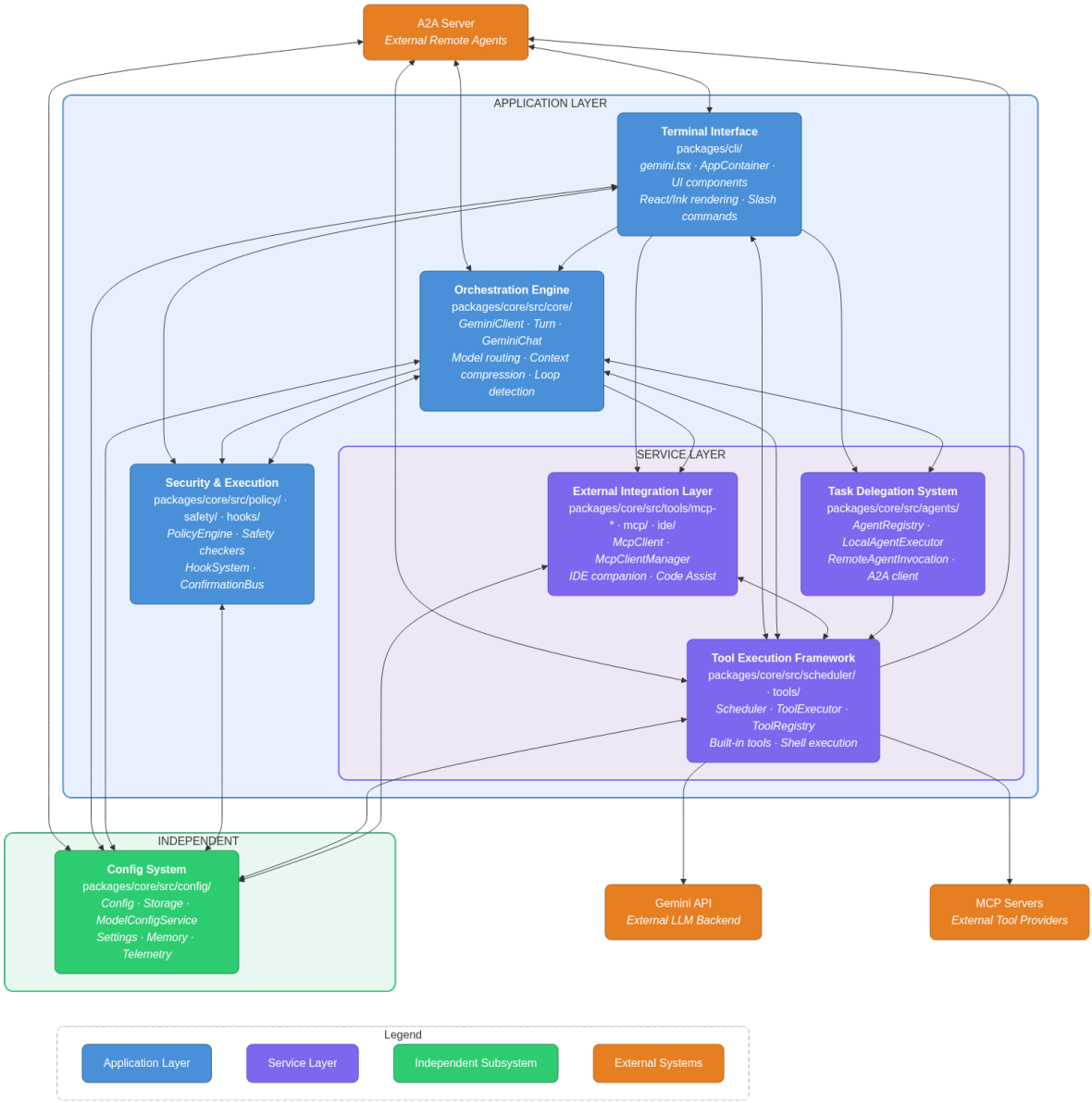

Figure 3: Concrete system architecture. Double-headed arrows indicate bidirectional coupling not predicted by the conceptual model.

Key Subsystems and Dependencies.

Terminal Interface ↔ Orchestration Engine (1300). The heaviest edge in the system. The Terminal Interface (`packages/cli`) consumes the Orchestration Engine's chat loop via `GeminiChat`, its `Turn` event stream, and shared utility modules. The Orchestration Engine (`core/src/core/`, `routing/`, `services/`) consumes UI-layer event types for streaming coordination. The two components share extensive infrastructure including error handling, event systems, and type definitions.

Orchestration Engine ↔ Tool Execution Framework (149 / 440). The Orchestration Engine drives tool execution through `coreToolScheduler`, which bridges into the `Scheduler` and `ToolExecutor`. The reverse dependency (440 imports) is driven by `coreToolHookTriggers` and the tool lifecycle's need for logging, context, and hook infrastructure that resides in the Orchestration Engine.

Task Delegation System → Tool Execution Framework (68). The agent system (`core/src/agents/`) reuses the tool infrastructure via `ToolRegistry.getTool()` and `executeToolCall()`. This is unidirectional: the Tool Execution Framework has no knowledge of agents.

Terminal Interface → Service Layer (220 + 168). The UI directly imports from the Tool Execution Framework (220) and External Integration Layer (168). React hooks such as `useToolScheduler` bridge tool schedul-

ing state into the rendering loop, while components like `McpStatus` render MCP server state. These edges represent a deliberate design choice where the UI consumes Service Layer state directly for rendering.

Security & Execution (cross-cutting). Has edges to every layer: 344/61 to Terminal Interface (confirmation dialogs rendered in the UI), 56/8 to Orchestration Engine (policy checks during content generation), 42/77 to Tool Execution Framework (per-tool policy evaluation), and 120/860 to Config System (policy rules loaded from configuration). The 860 reverse edge into Config reveals deep entanglement between policy definitions and configuration validation.

Config System (cross-cutting). Universally depended upon with incoming edges from every component. Encompasses settings schema (`settingsSchema.ts`), model configurations (`core/src/config/`), the hooks system (`core/src/hooks/`), and extension management (`cli/src/config/extension-manager.ts`).

External Boundaries. The Gemini API is consumed exclusively by the Orchestration Engine via `baseLlmClient` and `GeminiChat.sendMessage()`. MCP Servers are accessed through `McpClient` in the External Integration Layer. The A2A Server is well-isolated with minimal coupling (71/4 to Config).

2.3 Component Breakdown

2.3.1 Terminal Interface

What it represents: The Terminal Interface is made up of the entire user-facing presentation layer, consisting mainly of `packages/cli/src/gemini.tsx`, `packages/cli/src/nonInteractiveCli.ts`, and the `packages/cli/src/ui/` directory. It captures raw user input (both keyboard and piped stdin) and includes all interactive rendering components built with React and Ink.
Why it exists: Architecturally, this component exists to decouple input/output operations and terminal rendering concerns from the core backend logic. By consisting of these responsibilities, the Orchestration Engine can operate completely independent of the presentation medium, allowing the switching between interactive and non-interactive operational modes while enabling future extensibility to alternative interfaces.

2.3.2 Orchestration Engine

What it represents: Residing within `packages/core/src/core/`, the Orchestration Engine serves as the authoritative central coordinator of the agentic loop. Key files include `client.ts` (the `GeminiClient`), `turn.ts`, and `geminiChat.ts`. It governs the application's state machine: capturing user prompts, invoking the language model, processing streaming responses, detecting tool calls, and orchestrating subsequent agentic turns.
Why it exists: The agentic loop pattern requires a central brain to manage the complex lifecycle of iterative prompt-response and tool-execution cycles. By centralizing loop-management logic, model routing, and context compression here, the system avoids scattering the specific, stateful flow control logic across different UI or tool layers; thus, ensuring high cohesion and maintainability.

2.3.3 Tool Execution Framework

What it represents: Located primarily within `packages/core/src/scheduler/` and `packages/core/ src/tools/`, this framework is the execution engine for all capabilities available to the model. It includes the `Scheduler`, `ToolExecutor`, and a central `ToolRegistry`, with concrete implementations for the multiple built-in tools (such as file manipulation and shell execution).
Why it exists: Tools provide the AI agent with mechanisms to interact with its external environment. This framework exists to establish a uniform, robust lifecycle for tool invocation, encompassing validation, user confirmation checking, and execution independent of whether the tool is natively built-in or dynamically discovered. This standardizes tool handling and enforces systematic execution.

2.3.4 External Integration Layer

What it represents: This layer, comprising `packages/core/src/tools/mcp-*`, `packages/core/src/ mcp/`, and `packages/core/src/ide/`, handles communication with auxiliary external services. It manages Model Context Protocol (MCP) clients [5, 8], OAuth capabilities, and integrations with IDE companions (such as the VS Code

plugin).
Why it exists: Employing the Adapter design pattern, this layer summarizes the complexities of out-of-process communication. It normalizes remote capabilities so that the core Orchestration Engine can interact with remote MCP servers or IDE contexts using the same programmatic interfaces as local built-in tools, enhancing extensibility without increasing core coupling.

2.3.5 Task Delegation System

What it represents: Defined within `packages/core/src/agents/`, this system encompasses the `AgentRegistry`, `LocalAgentExecutor`, and A2A (Agent-to-Agent) networking logic. It is responsible for parsing agent definitions and launching isolated sub-agents to handle specialized procedures and workflows.
Why it exists: Complex overarching goals often require decomposition into specialized sub-tasks. The Task Delegation System provides functional encapsulation by assigning discrete context windows, specific tool subsets, and bounded turn resources to sub-agents. Additionally, it allows both local delegation and remote, cross-infrastructure collaboration via the A2A protocol [9].

2.3.6 Security & Execution

What it represents: This subsystem governs the security posture and event interception of the platform, localized in `packages/core/src/policy/`, `packages/core/src/safety/`, and `packages/core/ src/hooks/`. It encompasses the declarative TOML-based `PolicyEngine`, content safety checkers, and a detailed hook system for evaluating executions before they happen.
Why it exists: Given the obvious risks of autonomous agents capable of modifying files or executing shell commands, a strict, centralized security model is essential. This component intercepts and evaluates all prospective tool actions against explicit user-defined policies, enforcing a security-by-default posture while allowing comprehensive enterprise customization without needing structural modifications to the core Orchestration Engine.

2.3.7 Config System

What it represents: Operating as the foundational configuration backbone, this system is primarily instantiated in `packages/core/src/config/` through the central `Config` class, the `Storage` subsystem, and model configurations. It persists user settings across sessions, manages scoped context for sub-agent loops, and resolves runtime variables.
Why it exists: Configuration state is present across all application layers. Instead of relying on decentralized environment variables or fragmented configuration files, the Config System centralizes this data. This guarantees a single source of truth, also permits precise dependency injection into localized components, and significantly enhances component testability and predictability.

2.4 Architectural Dependencies

The structural integrity and operational flow of the Gemini CLI relies on fundamental data and control dependencies between these components. Using the import counts from Figure 2, we now detail the most critical dependencies at the method level:

- **Terminal Interface ↔ Orchestration Engine (1300 imports):**
  - **What it is:** The `AppContainer` component in the Terminal Interface explicitly creates instances of and calls `sendMessageStream()` on the `GeminiClient`. The UI then consumes the resulting `AsyncGenerator` stream to render text, tool-call progress, and completions. The reverse direction arises from shared infrastructure including error handling, event systems, and type definitions.
  - **Why it is necessary:** This bidirectional coupling is the heaviest edge in the system. The Terminal layer is the primary consumer of the Orchestration Engine, establishing the core operational pipeline where captured user input is forwarded to the executing engine and real-time asynchronous stream segments are returned for interactive UI rendering.
- **Orchestration Engine ↔ Tool Execution Framework (149 / 440 imports):**

- **What it is:** In the forward direction, when the `GeminiClient` detects a `functionCall` requested by the model, it passes a `ToolCallRequestInfo` array to the `Scheduler` via the `schedule()` method, yielding an array of `CompletedToolCall` results. The heavier reverse direction (440 imports) is driven by `coreToolHookTriggers` and the tool lifecycle's need for logging, context, and hook infrastructure that resides in the Orchestration Engine.
- **Why it is necessary:** While the language model dictates intent, it cannot execute actions directly. This dependency is required to physically translate the model's synthesized tool requests into concrete, validated actions. The reverse coupling, though unexpected in the conceptual model, arises from two bridge files (`coreToolScheduler.ts` and `coreToolHookTriggers.ts`) that implement tool execution logic but reside in the Orchestration Engine's directory.

- **Tool Execution Framework → Security & Execution (42 / 77 imports):**
  - **What it is:** Before any registered tool is allowed to execute, the internal `Scheduler._processToolCall()` method explicitly consults the `PolicyEngine` (by invoking `PolicyEngine.check()`) to derive an authorization decision (`ALLOW`, `DENY`, or `ASK_USER`).
  - **Why it is necessary:** This dependency is important for enforcing the system's runtime security constraints. It ensures that absolutely no operative capability bypasses authorization policies, maintaining rigid control over potentially destructive or sensitive actions.

- **Terminal Interface → Service Layer (220 + 168 imports):**
  - **What it is:** The UI directly imports from the Tool Execution Framework (220) and the External Integration Layer (168), bypassing the Orchestration Engine. React hooks such as `useToolScheduler` bridge tool scheduling state into the rendering loop, while components like `McpStatus` render MCP server connection status.
  - **Why it is necessary:** The Terminal Interface must render tool execution progress, confirmation prompts, and MCP server status in real time. Rather than routing this state through the Orchestration Engine as a mediator, the UI subscribes directly to the Scheduler's state machine and the MCP client's connection status. This is a pragmatic design choice that favors rendering performance over strict layering.

- **External Integration Layer → MCP Servers (External):**
  - **What it is:** The `McpClient.connect()` routine establishes a connection to a remote Model Context Protocol server, immediately followed by calls to `discover()` endpoints (such as `tools/list`) to dynamically resolve external capabilities into the local `ToolRegistry`.
  - **Why it is necessary:** Relying exclusively on predefined functions would restrict the agent. This outbound dependency is essential because it permits the underlying application capability to scale dynamically, granting the agent interoperability with third-party extensions and enterprise data sources at runtime.

3 Tool Execution Framework

The Tool Execution Framework is responsible for the complete lifecycle of every tool call the AI model produces, from the moment a `functionCall` is detected in a model response to the moment a `functionResponse` is returned to the next prompt. It spans three source directories: `src/scheduler/`, `src/tools/`, and `src/policy/`, along with the messaging infrastructure in `src/confirmation-bus/`.

3.1 Conceptual Architecture

At the conceptual level, the framework decomposes into five sub-components organized in a **hub-and-spoke** topology, where the Scheduler serves as the central orchestrator.

1. **Scheduler:** The event-driven orchestrator that drives the lifecycle of a batch of tool calls through ingestion, policy checking, confirmation, execution, and completion phases.

2. **Tool Executor:** A focused execution unit that invokes a tool's `.execute()` method, manages output streaming, handles cancellation, truncates excessive output, and formats the result into `functionResponse` parts for the model.

3. **Tool Registry:** The canonical registry for all tool definitions (built-in, discovered, and MCP), implementing the Registry pattern to decouple tool consumers from tool providers.

4. **Tool Definitions:** The base classes, interfaces, and concrete tool implementations that define what each tool does, how its parameters are validated, and what confirmation UI it presents.

5. **Policy/State Manager:** A dual-responsibility component comprising the PolicyEngine (security rule evaluation) and the SchedulerStateManager (tool call lifecycle state tracking), along with the MessageBus infrastructure for confirmation flows.

Dependency Direction. Dependencies flow **outward from the Scheduler**. As seen in Figure 4, the Scheduler depends on all four other sub-components, but none of them depend back on the Scheduler. This hub-and-spoke pattern ensures that the orchestration logic can be modified independently of policy rules, tool implementations, or execution mechanics.

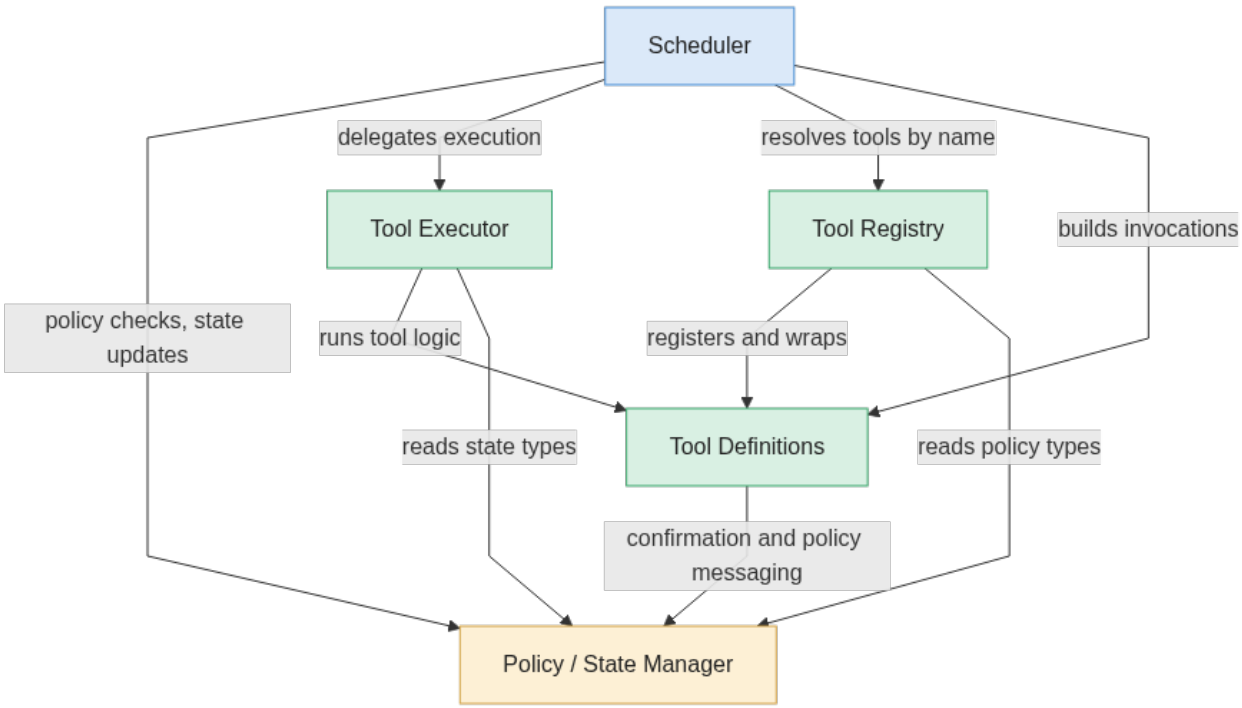

Figure 4: Conceptual dependency graph of the Tool Execution Framework. Arrows indicate the direction of dependency (caller → callee).

3.2 Concrete Architecture

3.3 Sub-Component Interactions

1. Scheduler → Tool Registry. When the Scheduler receives a batch of `ToolCallRequestInfo` objects, its `_startBatch()` method calls `ToolRegistry.getTool(name)` to resolve each tool name to a concrete `DeclarativeTool` instance. It then calls `tool.build(args)` to create a validated `ToolInvocation` object. This is a unidirectional dependency: the Registry never calls back into the Scheduler.

2. Scheduler → Policy/State Manager. For each tool call, the Scheduler invokes `checkPolicy(toolCall, config)`, which delegates to `PolicyEngine.check()`. The PolicyEngine evaluates the call against a prioritized list of `PolicyRule` objects and returns one of three decisions: `ALLOW`, `DENY`, or `ASK_USER`. If the decision is `ASK_USER`, the Scheduler calls `resolveConfirmation()`, which publishes a request to the `MessageBus` and waits for the user's

response. Throughout this process, the Scheduler updates the `SchedulerStateManager` via `updateStatus()` calls, which enforce the following state machine:

```
Validating --> Scheduled        (policy ALLOW)
Validating --> AwaitingApproval (policy ASK_USER)
Validating --> Error            (policy DENY)
AwaitingApproval --> Scheduled  (user approves)
AwaitingApproval --> Cancelled  (user denies)
Scheduled  --> Executing        (execution begins)
Executing  --> Success          (tool returns)
Executing  --> Error            (tool throws)
Executing  --> Cancelled        (user aborts)
```

3. Scheduler → Tool Executor. Once a tool call reaches the `Scheduled` state, the Scheduler delegates to `ToolExecutor.execute()`, passing a `ToolExecutionContext` that includes the tool call, an `AbortSignal`, and output streaming callbacks. The Tool Executor calls `invocation.execute(signal, updateOutput)`, processes the `ToolResult`, truncates output if needed, and returns a `SuccessfulToolCall` or `ErroredToolCall`.

4. Tool Executor → Tool Definitions. The Tool Executor's sole interaction with Tool Definitions is through the `ToolInvocation` interface, specifically its `execute()` method. The Executor is agnostic to which concrete tool is running; it operates entirely through the polymorphic interface defined in `tools.ts`.

5. Tool Definitions → Policy/State Manager. This is the most significant cross-cutting dependency. The base class `BaseToolInvocation` in `tools.ts` contains `getMessageBusDecision()`, which publishes a `ToolConfirmationRequest` to the `MessageBus` and subscribes for the `ToolConfirmationResponse`. It also contains `publishPolicyUpdate()`, which publishes `UPDATE_POLICY` messages when users select "always allow" options. Individual tools like `edit.ts` and `shell.ts` also import `ApprovalMode` from `policy/types.ts` to determine their confirmation behavior. This coupling is the primary source of bidirectional edges in the dependency graph.

3.4 Concrete Dependency Analysis

Using SciTools Understand, we generated a dependency graph for the five sub-components (Figure 5). The edge labels represent import counts between components.

Each conceptual sub-component maps to specific source files in the codebase. To understand the concrete interactions between these components, we used SciTools Understand to generate a dependency graph (Figure 5) mapping the actual import counts between the five sub-components.

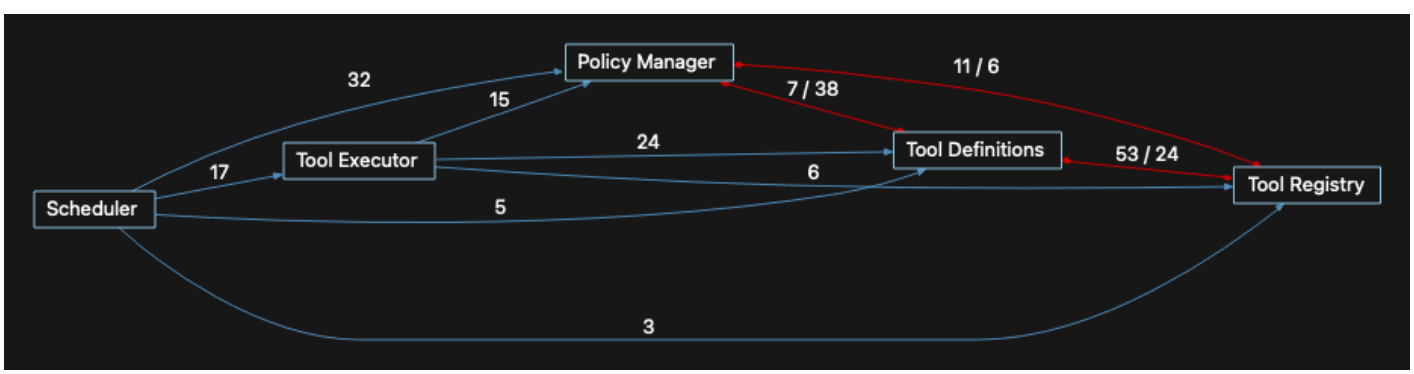

Figure 5: Dependency graph generated by SciTools Understand. Blue edges are unidirectional; red edges are bidirectional. Edge labels indicate import counts.

From this dependency analysis, we derived the concrete architecture diagram (Figure 6), which annotates each edge with the specific method calls and type imports that create the dependency.

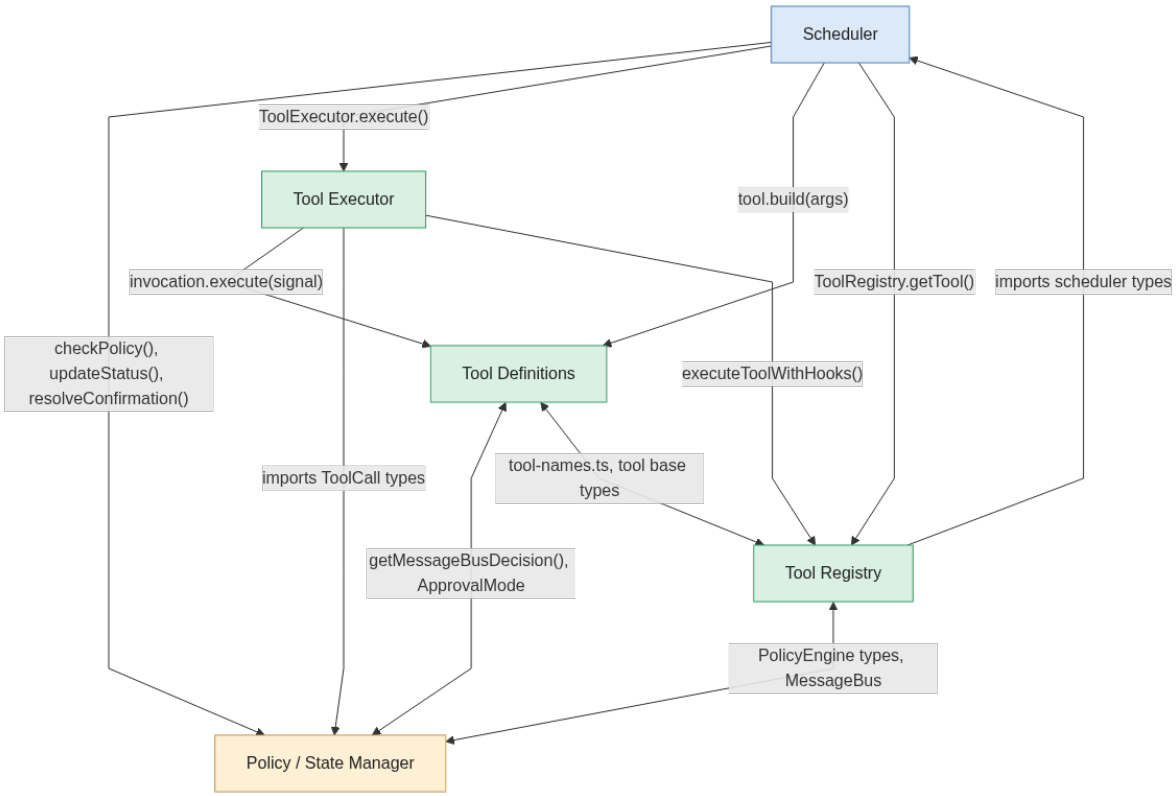

Figure 6: Concrete architecture of the Tool Execution Framework, derived from the dependency graph. Edge labels show the specific method calls and type imports that create each dependency.

Key Observations:

- The **Scheduler** exhibits clean, unidirectional outbound edges to all four other sub-components (32→Policy Manager, 17→Tool Executor, 5→Tool Definitions, 3→Tool Registry), confirming the hub-and-spoke topology described in the conceptual architecture.
- The **Tool Executor** also has clean unidirectional edges (15→Policy Manager, 24→Tool Definitions), validating that it is a pure downstream consumer with no reverse dependencies.
- The heaviest edge is **Tool Definitions ↔ Tool Registry** (53/24), driven by concrete tools importing `tool-names.ts` constants from the Registry, and the Registry importing tool base types for registration.
- The **Tool Definitions ↔ Policy/State Manager** edge (7/38) reflects the architectural coupling where `BaseToolInvocation` directly interacts with the MessageBus and policy types for confirmation flows. This bidirectional dependency is the primary candidate for future refactoring.

4 Use Cases and Execution Traces

This section traces two distinct interaction scenarios through the concrete architecture, demonstrating the flow of control and data using actual method calls. By examining these traces, we can observe how the system handles different policy constraints and user interactions.

4.1 Use Case 1: Read-Only Tool Execution (`read_file`)

In this scenario, the user asks a question that prompts the model to issue a `read_file` tool call. Because this operation is non-destructive, the `PolicyEngine` automatically approves it, allowing the execution to proceed without interrupting the user.

The execution trace in Figure 7 begins when the `TerminalInterface` calls `sendMessageStream()` to dispatch the user's prompt to the `OrchestrationEngine`. Once the model's response reveals a `functionCall` [10, 4], the

Orchestration Engine invokes `schedule()` on the `Scheduler`. During the processing loop, the `Scheduler` invokes `checkPolicy()` to consult the `PolicyEngine`, which recognizes the non-destructive nature of the tool and returns an `ALLOW` decision. Consequently, the `StateManager` immediately transitions the call to `Scheduled` and then `Executing`, bypassing any confirmation dialogs. Finally, the `ToolExecutor` delegates to `executeToolWithHooks()`, reading the file and returning a
`SuccessfulToolCall` to the Orchestration Engine for the subsequent processing turn.

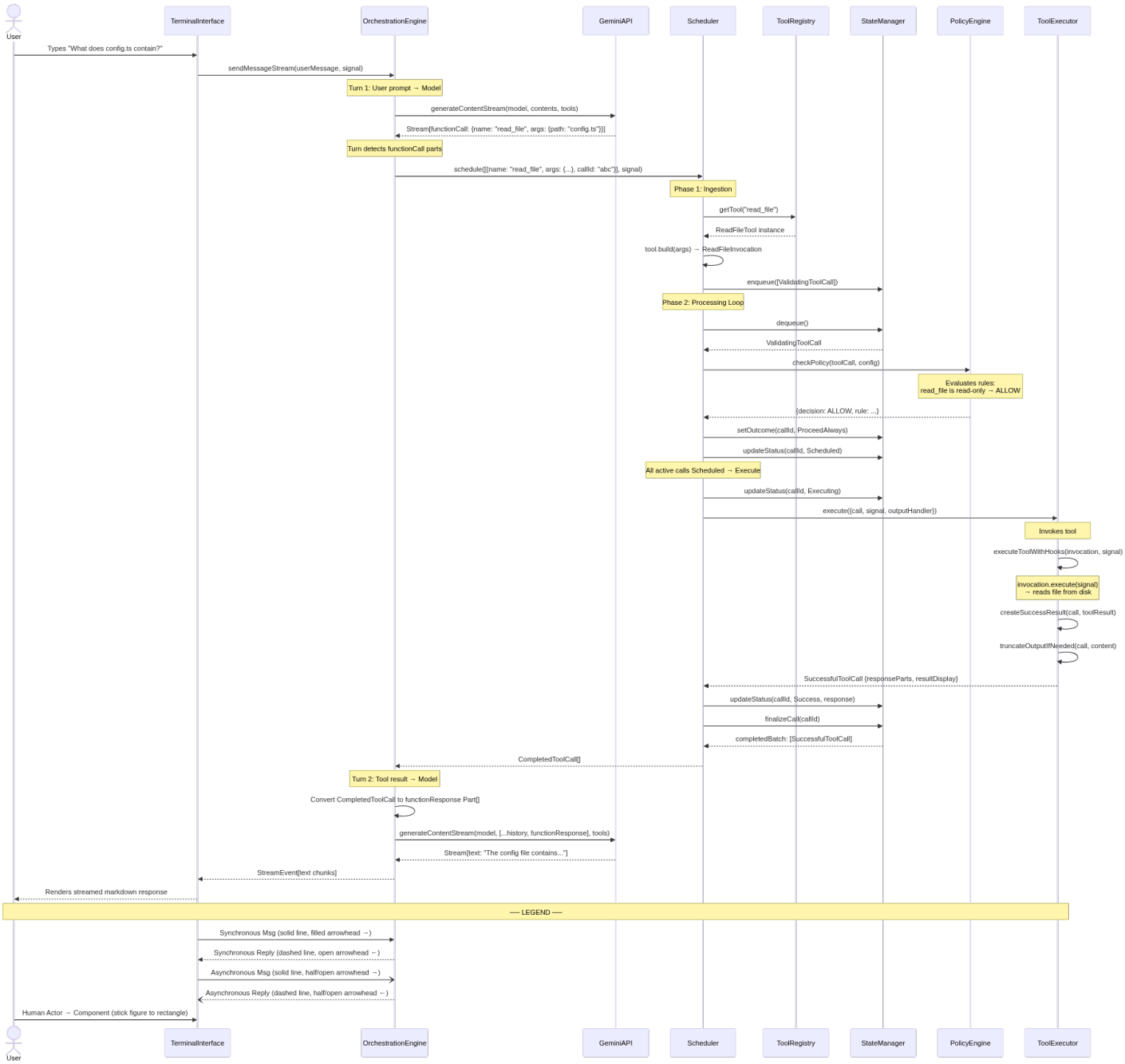

Figure 7: Execution trace for a read-only tool call, illustrating the automated `ALLOW` evaluation by the `PolicyEngine`.

4.2 Use Case 2: Safety-Gated Code Modification (`write_file`)

In this scenario, the user requests a code modification that triggers a `write_file` tool call. Because this is a destructive operation, the `PolicyEngine` flags it with an `ASK_USER` decision. The framework pauses the execution to request explicit user confirmation via the `MessageBus` before proceeding.

The initial ingestion in Figure 8 proceeds identically to the first use case until the `Scheduler` evaluates the call

via `checkPolicy()`. At this juncture, the `PolicyEngine` issues an `ASK_USER` decision. The `Scheduler` responds by executing the `resolveConfirmation()` routine, transitioning the call state to `AwaitingApproval`, and publishing a `TOOL_CALLS_UPDATE` event to the `MessageBus`. This event prompts the `TerminalInterface` to render an interactive approval dialog displaying the proposed file differences. Once the user clicks "Approve", a `TOOL_CONFIRMATION_RESPONSE` is dispatched back to the framework, resolving the confirmation loop. Only then does the `Scheduler` permit the `ToolExecutor` to proceed with `executeToolWithHooks()`, ensuring that the destructive write completes safely and securely.

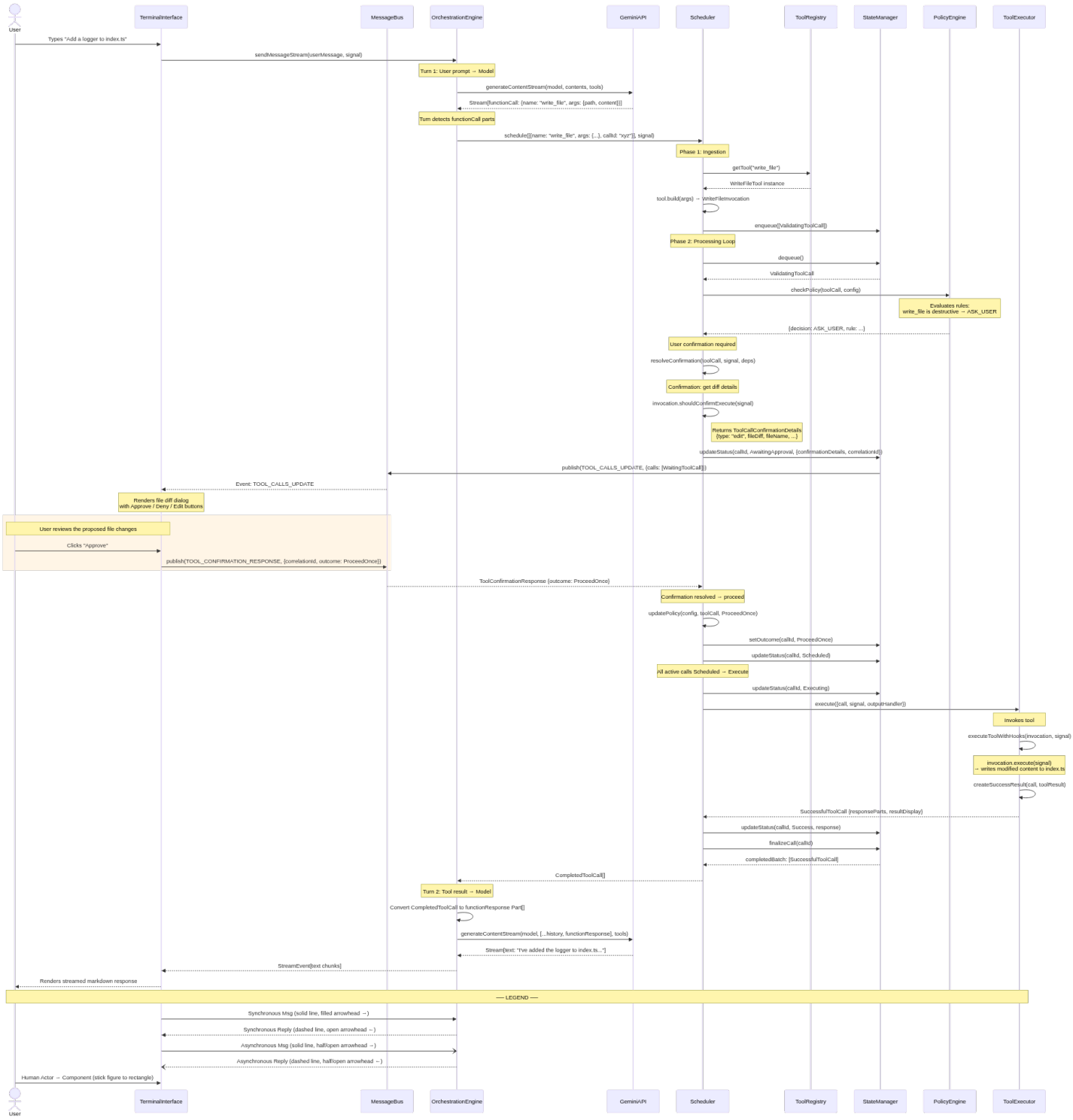

Figure 8: Execution trace for a safety-gated tool call, highlighting the Authorization Gate, `ASK_USER` policy decision, and the interactive `MessageBus` confirmation loop.

5 Reflexion Analysis

This section analyzes the discrepancies between our initial A1 conceptual model and the actual concrete implementation. By anchoring our evaluation in factual evidence extracted directly from the codebase, we uncover how real-world engineering constraints shaped the architectural evolution of the Gemini CLI system.

5.1 High-Level Architectural Divergences

The concrete architecture reveals significant divergences from the conceptual model established in A1, spanning from overarching paradigm shifts to specific structural couplings. For each major finding, we contrast our initial assumptions with the concrete reality, provide the engineering rationale, and describe how the architectural model was updated.

5.1.1 Client/Server Split vs. Layered Monorepo

- **Conceptual View (A1):** We modeled the system using a Client/Server architecture, implying a strict network boundary separating a client-side UI from a remote server-side backend.
- **Concrete Reality (A2):** The system operates as a single-process monorepo. It utilizes npm workspaces where `packages/cli` directly imports TypeScript modules from `packages/core`. The system is fundamentally a thick client communicating exclusively with remote APIs (e.g., Gemini API, remote MCP servers), rather than a disjointed client/server pair.
- **Engineering Rationale:** A terminal tool demands sub-second responsiveness. Forcing a network hop between the UI and the backend would introduce unacceptable latency for streaming tokens and tool-call updates. Additionally, direct TypeScript imports provide rigorous compile-time type safety for complex data structures across boundaries.
- **Update to architecture:** The updated concrete diagram removes the network boundary distinction, consolidating the components into a unified, single-process system model.

5.1.2 Strict Layered Stack vs. Bypassed Mediation

- **Conceptual View (A1):** Our model assumed a vertical, top-down three-layer stack. We assumed strict layered access where the Terminal Interface would interact with underlying tools *only* through the Orchestration Engine acting as a mediator.
- **Concrete Reality (A2):** The architecture actually utilizes horizontal layering where the Terminal Interface heavily bypasses the Orchestration Engine. The concrete dependency graph reveals direct edges from the Terminal Interface to the Tool Execution Framework (220 imports) and the External Integration Layer (168 imports).
- **Engineering Rationale:** The React/Ink UI must render tool execution progress, confirmation prompts, and MCP server status in real time. Rather than routing this state through the Orchestration Engine as a bottleneck, the UI subscribes directly to the Scheduler's state machine and the MCP client's connection status. This is a pragmatic design choice that favors rendering performance over strict layering.
- **Update to architecture:** The concrete diagram adds direct edges from the Terminal Interface to all three Service Layer components, reflecting the UI's role as a direct consumer of service-level state.

5.1.3 Implicit Security vs. Entangled First-Class Subsystems

- **Conceptual View (A1):** Security was largely implicit, and the Config System was modeled as a simple, independent cross-cutting component that is passively consumed by other layers.
- **Concrete Reality (A2):** Security is actually a standalone, first-class subsystem (governed by the `PolicyEngine`). However, the concrete architecture reveals that Security & Execution and the Config System are deeply entangled, sharing a massive 120/860 bidirectional edge, far heavier than any edge to the components they are supposed to serve.

- **Engineering Rationale:** This entanglement exists because security *policies* are defined as configuration (TOML files loaded via the config system), while configuration *validation* depends on security rules (trusted folders, sandbox profiles). The two components are co-dependent by design.
- **Update to architecture:** The concrete diagram adds a heavy bidirectional edge between Security & Execution and the Config System, and repositions both as vertical slices that span the layered hierarchy rather than occupying a fixed horizontal layer.

5.1.4 Tightly Coupled Orchestration and Tool Execution

- **Conceptual View (A1):** The conceptual model depicted a clean, unidirectional caller-callee relationship: the Orchestration Engine schedules tools, and the tools return results.
- **Concrete Reality (A2):** The concrete architecture shows that the Orchestration Engine and Tool Execution Framework form a tightly coupled pair with a 149/440 bidirectional edge, with the reverse direction (440) being the heavier side.
- **Engineering Rationale:** This coupling originates from two bridge files: `coreToolScheduler.ts` and `coreToolHookTriggers.ts`, that reside in the Orchestration Engine's directory but implement Tool Execution Framework logic. The tool lifecycle requires access to the Orchestration Engine's logging, hook system, and context management infrastructure, creating a massive reverse dependency.
- **Update to architecture:** The concrete diagram replaces the unidirectional Orchestration→Tools arrow with a bidirectional edge, reflecting the tight integration between the chat loop and the tool execution pipeline.

5.1.5 Confirmed Architectural Boundaries (Unchanged from A1)

While the codebase revealed several unexpected couplings, two aspects of our conceptual architecture were confirmed to be accurate and remain unchanged:

- The **Task Delegation System** maintains a clean unidirectional dependency on the Tool Execution Framework (68 imports, no reverse edge), confirming the conceptual model's separation of concerns between agent delegation and tool execution.
- The **A2A Server** remains well-isolated (71/4 to Config, minimal edges elsewhere), validating its conceptual design as an independent external boundary.

5.2 Subsystem Reflexion: Tool Execution Framework.

The Tool Execution Framework exemplifies the gap between our initial conceptual understanding and the actual codebase. In our first-pass conceptual model, we envisioned a flat "Tools Layer" managed by a simple "Tool Manager" that linearly invoked static functions. The concrete architecture revealed a far more sophisticated design: a 5-component framework comprising a `Scheduler`, `ToolExecutor`, `ToolRegistry`, `Tool Definitions`, and `PolicyEngine`, organized in a hub-and-spoke topology.

The dependency analysis using SciTools Understand confirmed that the Scheduler operates as a clean hub with unidirectional outbound edges to all other components. However, it also exposed coupling that our conceptual model did not predict, particularly bidirectional dependencies between Tool Definitions and the Policy/State Manager, where confirmation and policy logic is embedded directly in tool base classes. This taught us that conceptual diagrams, while useful for high-level reasoning, can mask significant structural complexity that only becomes visible through static analysis tooling.

More broadly, multiple complexities emerged in the concrete architecture that our initial model failed to anticipate:

Runtime Extensibility: Rather than a static list of tools, the `ToolRegistry` dynamically registers and resolves implementations, seamlessly integrating local functions alongside dynamically discovered MCP tools.

Lifecycle State Management: Simple synchronous invocation has been replaced by the `Scheduler`, which drives calls through a formal state machine (`Validating` → `Scheduled` → `Executing`), enabling parallel execution of read-only capabilities.

Security and User Confirmation: The pipeline includes mandatory evaluation by the `PolicyEngine` and supports an asynchronous, user-in-the-loop confirmation phase (`ASK_USER`), allowing users to pause execution to inspect or modify tool arguments.

5.3 Summary of Rationales

In conclusion, the concrete architecture diverges from our initial conceptual models to prioritize robust execution safety, critical latency reduction, and runtime extensibility. By abandoning a strict vertical hierarchy in favor of event-driven hubs and independent, gated subsystems, the Gemini CLI ensures it remains responsive as a terminal utility while safely executing complex, potentially destructive autonomous actions.

6 Lessons Learned

Our most profound technical lesson emerged from contrasting our initial architectural assumptions against the concrete reality of the codebase. Initially, we envisioned a simple "Tools Layer" where tools were invoked synchronously and linearly by a monolithic core manager. However, dissecting the Tool Execution Framework revealed a hyper-complex, event-driven, hub-and-spoke ecosystem. We vastly underestimated the real-world complexity required to safely execute non-deterministic LLM outputs on a local machine. Discovering the non-bypassable `PolicyEngine` gate and the asynchronous user-in-the-loop confirmation loops taught us that enterprise-grade AI systems prioritize defense-in-depth and lifecycle state management far above the simple function-calling tutorials often depicted online.

Procedurally, this project highlighted the practical realities of leveraging local AI agents for architectural discovery [6]. We initially assumed we could simply instruct our AI coding assistant to ingest the entire monorepo and output a unified architecture diagram. However, this naive approach resulted in `unexpected EOF` stream reading timeouts and context window exhaustion. We quickly learned to treat the AI as a focused collaborator rather than a magic oracle. By "chunking" our architectural queries, asking it to exclusively trace import boundaries for one specific directory at a time, and heavily constraining its diagramming output with strict directional dependency rules, we prevented the generation of incoherent "spaghetti" architectures and produced highly accurate, targeted subsystem graphs.

Finally, from a project management perspective, this assignment underscored the danger of delaying concrete analysis. By beginning our codebase exploration and tool-assisted tracing early, we afforded ourselves the necessary iterative cycles to refine our AI prompts and validate the generated findings manually against the actual repository structure. This proactive workflow prevented the panic of compiling a rushed, hallucinated architecture report at the last minute, allowing us to deliver a factual, deeply analytical reflexion on the Gemini CLI system.

A AI Collaboration Report

AI Member Profile and Selection Process

For this deliverable, our group utilized a dual-AI teammate approach, pairing **Google Gemini 3.1 Pro** (Web Interface) with a Local Multi-Agent System (powered by Claude Code). This dual setup was selected for two strategic reasons:

- **Architectural Codebase Ingestion:** Recovering the as-built concrete architecture required analyzing thousands of lines of code across a monorepo. Standard web-based LLMs cannot easily ingest entire local repositories without exceeding context windows or requiring tedious manual copy-pasting. The Local Agent could natively traverse our cloned `gemini-cli` repository, trace import graphs, and read directory structures (`packages/cli` vs. `packages/core`).
- **Meta-Prompting and Rubric Alignment:** We utilized the Gemini Web interface as a Senior Prompt Engineer. By uploading the A2 project specifications and the grading rubric, we used Gemini Web to engineer highly constrained prompts that guaranteed the Local Agent's outputs would strictly align with the "Great" column of the rubric.

Tasks Assigned to the AI Teammate

We treated the AI suite as a combination of an Architectural Consultant and a Codebase Analyst. The specific tasks delegated were:

- **Iterative Subsystem Mapping:** We tasked the Local Agent with scanning the repository and mapping the actual source code files to the 7-component reference architecture provided on onQ.
- **Subsystem Deep Dive:** We assigned the AI to decompose the Tool Execution Framework into at least four sub-components (Scheduler, ToolExecutor, ToolRegistry, PolicyEngine) and trace their exact concrete dependencies.
- **Reflexion Analysis Generation:** We tasked the AI with comparing our A1 conceptual assumptions (a strict Client/Server UI/Core/Tools stack) against the A2 concrete reality (a thick-client layered monorepo), extracting the engineering rationales for these divergences.
- **Use Case Sequence Tracing:** We assigned the AI to trace two use cases through the codebase, outputting the execution flow using concrete method calls mapped into Mermaid.js syntax for easy diagram generation.

Interaction Protocol and Prompting Strategy

Our group utilized a Two-Tiered Meta-Prompting Protocol combined with a Chunked Execution Strategy.

- **Tier 1 (Strategy Definition):** We first interacted with Gemini Web, feeding it the grading rubric and asking it to act as an expert prompt engineer. We explicitly instructed it to formulate prompts that would extract the what and why for architectural components.
- **Tier 2 (Local Execution & Chunking):** We pasted these engineered prompts into the Local Agent. During our initial high-level mapping, the Local Agent encountered an unexpected EOF stream reading error because the context load of scanning the entire monorepo at once caused an API timeout. We immediately iterated our prompting strategy by "chunking" the requests: we prompted the agent to only analyze the Application Layer first, wait for confirmation, and then proceed to the Service Layer.
- **Example Deep-Dive Prompt:** To prevent the AI from generating a "Big Ball of Mud" dependency graph, we used strict constraints: "Explain exactly what each of these 4 sub-components does internally... Trace the exact concrete dependencies... Crucially, identify the direction of the dependencies (e.g., The Scheduler calls the ToolRegistry, not the other way around)."

Validation and Quality Control Procedures

Because AI models can hallucinate dependencies or misunderstand architectural patterns, we implemented a rigorous validation protocol:

- **Architectural Sanity Checks:** Prior to using the AI, a human teammate manually drafted a dependency diagram for the "Core" subsystem featuring bidirectional arrows between all components. We used the AI to validate this approach and subsequently rejected the human draft, as the AI correctly identified through code tracing that the Tool Execution Framework operates on a strict, unidirectional "hub-and-spoke" model.
- **Source Code Verification:** When the AI generated the concrete mapping, we did not blindly accept its claims. When it stated that the `McpClient` managed external integrations or that the `PolicyEngine` was a standalone security gate, we manually opened `packages/core/src/tools/ mcp-client.ts` and `packages/core/src/policy/policy-engine.ts` in our IDE to verify the class responsibilities and import statements.
- **Method Call Tracing:** For the sequence diagrams, we verified that the method calls generated by the AI in the Mermaid code (e.g., `sendMessageStream()`, `checkPolicy()`) actually existed in the codebase and matched the documented control flow.

Quantitative Contribution to Final Deliverable

We estimate the AI tools contributed approximately **40%** to the final deliverable. This breaks down as:

- **15%** for extracting the concrete codebase mapping and grouping files into subsystems.
- **10%** for drafting the initial comparisons and engineering rationales for the Reflexion Analysis.

- **10%** for generating the Mermaid.js code used to render our sequence diagrams.
- **5%** for rubric alignment, meta-prompt engineering, and identifying flaws in our initial human-drawn diagrams.

Reflection on Human-AI Team Dynamics

Integrating a dual-AI workflow profoundly shifted our team dynamics from manual code digging to high-level architectural curation. The Local Agent acted as an incredible force multiplier; tasks that would normally take hours of manually tracing import statements across a monorepo were completed in minutes.

However, this workflow introduced unique technical challenges. The API stream timeouts taught us that AI agents have hard systemic limits, requiring us to adapt our communication style to break complex architectural queries into digestible, sequential chunks. Furthermore, we learned that while the AI is exceptional at finding concrete code paths, humans must still provide the architectural guardrails. If we had not explicitly constrained the AI to focus on specific layers and directional flows, it would have generated an overwhelming, unreadable map of the codebase. Effective AI collaboration in software architecture requires treating the AI as a powerful code-reading engine that still relies entirely on the human to define the structural constraints.

References

[1] J. Alateras, "Unpacking the gemini CLI: A high-level architectural overview," *Medium*, Jul. 8, 2025. [Online]. Available: https://medium.com/@jalateras/unpacking-the-gemini-cli-a-high-levelarchitectural-overview-99212f6780e7

[2] "Architecture Overview: Google-gemini/gemini-CLI," *DeepWiki*, Feb. 6, 2026. [Online]. Available: https://deepwiki.com/google-gemini/gemini-cli/1.1-architecture-overview

[3] "Gemini CLI Architecture Overview," *Gemini CLI*. [Online]. Available: https://geminicli.com/docs/architecture

[4] "Products and Services — Google Cloud," *Google*. [Online]. Available: https://cloud.google.com/products

[5] "What is the Model Context Protocol (MCP)?," *Model Context Protocol*. [Online]. Available: https://modelcontextprotocol.io/docs/getting-started/intro

[6] "Gemini CLI Github Releases," *GitHub*. [Online]. Available: https://github.com/google-gemini/gemini-cli/releases

[7] "Gemini CLI core: Tools API," *Gemini CLI*. [Online]. Available: https://geminicli.com/docs/core/tools-api/

[8] "Model Context Protocol." [Online]. Available: https://modelcontextprotocol.io/

[9] "Agent-to-Agent Protocol." [Online]. Available: https://a2a.dev/

[10] "Gemini API," *Google AI for Developers*. [Online]. Available: https://ai.google.dev/gemini-api/docs